Scalability & Reliability Design :

This document details how the architecture handles **10,000+ RPS burst spikes** while maintaining **zero overselling**, sub-100ms latency SLAs, and 99.99% availability.

Save these details in `07_Scalability_and_Performance/` and `06_Concurrency_and_Fault_Tolerance/`.

1. High-Concurrency Scalability Strategy
To scale seamlessly under high-volume burst conditions without crashing the backend, the platform implements a **multi-tiered traffic shedding and scaling pipeline**:

1.1 Edge Traffic Shaping & Virtual Queueing

- **Virtual Waiting Room (Cloudflare Edge):** Absorbs extreme incoming spikes (e.g., 50,000 concurrent connection hits at the exact start of a flash sale). Users are placed in a FIFO virtual queue at the edge and admitted to the API Gateway in controlled batches matching backend processing capacity.
- **Token-Bucket Rate Limiting:** Enforced at the API Gateway layer (Kong/Envoy) per IP and authenticated JWT ID to block automated bots, malicious scrapers, and brute-force retry scripts.

1.2 Horizontal Pod Autoscaling (HPA)

- Microservices run inside Kubernetes clusters across multiple Availability Zones.
- **Scaling Metrics:** HPA scales backend pods using custom Prometheus metrics (`http_requests_per_second` and `cpu_utilization_percent`).
- **Pre-Warming Policy:** 15 minutes before an announced flash sale event, a scheduled CronJob scales up the `Inventory Service` and `Order Service` replicas to their peak capacity (e.g., 50 pods) to avoid startup latency during warm-up periods.

1.3 Database Scaling & Read-Write Separation

- **Read-Write Splitting:** All write mutations (`INSERT` orders, `UPDATE` balances) hit the primary PostgreSQL write node, while read-heavy operations (order history lookups, product detail browsing) are routed across 3 Read Replicas.
- **Connection Pooling:** Services connect to PostgreSQL using **PgBouncer** in transaction pooling mode to prevent database thread exhaustion under high concurrent connection loads.

2. Reliability, Fault Tolerance & Resiliency
To prevent cascading system failures when downstream providers encounter degradation or outages, the architecture incorporates core fault-tolerance patterns:

2.1 Resiliency Pattern Matrix

Pattern	Implementation	Problem Solved
Circuit Breaker	Resilience4j wrapping external Payment Gateways (Stripe/Razorpay).	Opens circuit if gateway failure rate exceeds 50% over a 10-second window, preventing application thread starvation.
Bulkhead Isolation	Isolated execution thread pools per microservice.	Prevents a complete slowdown in the Payment Service from consuming memory or CPU resources allocated to Inventory Reservation.
Exponential Backoff & Jitter	Client and worker retry policies for transient network errors.	Prevents the "thundering herd" problem during automatic service recovery scenarios.
Graceful Degradation	Non-essential UI components (recommendation engines, reviews) are toggled off during flash sales.	Frees up network bandwidth and compute capacity exclusively for core inventory checkout execution paths.

3. SLA, Performance Targets & Benchmarks

The platform is designed and tested to strictly meet the following SLAs under load testing:

3.1 Performance SLA Metrics

Performance Metric	Target SLA	Strategy to Achieve SLA
Peak Throughput	**10,000+ RPS**	In-memory Redis cluster atomic stock processing + Kafka event buffering.

P95 Latency	**< 80 ms**	Non-blocking API execution path; DB reads offloaded to Redis cache.
P99 Latency	**< 150 ms**	Asynchronous order processing via background Kafka workers.
Stock Overselling Rate	**0.00% (Zero)**	Redis Lua single-threaded atomic checking & decrements (EVALSHA).
System Availability	**99.99% Uptime**	Multi-AZ deployment with automated Kubernetes failover.

4. k6 Load Testing Script Sample

Save this script as `07_Scalability_and_Performance/load_test.js` to simulate peak concurrency during benchmarking:

```
import http from 'k6/http';
import { check, sleep } from 'k6';
import { uuidv4 } from 'https://jslib.k6.io/k6-utils/1.4.0/index.js';

export const options = {
  stages: [
    { duration: '30s', target: 2000 },  // Ramp up to 2,000 VUs
    { duration: '1m',  target: 10000 }, // Spike to 10,000 VUs (Flash Sale Start)
    { duration: '2m',  target: 10000 }, // Hold peak 10,000 RPS load
    { duration: '30s', target: 0 },     // Ramp down
  ],
  thresholds: {
    http_req_failed: ['rate<0.01'],    // Error rate must be under 1%
    http_req_duration: ['p(95)<80'],   // 95% of requests must complete under 80ms
  },
};

export default function () {
  const url = 'https://api.salestorm.io/v1/reservations';
  const payload = JSON.stringify({
    product_id: '9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d',
    quantity: 1,
  });

  const params = {
```

```
    headers: {
      'Content-Type': 'application/json',
      'Authorization': 'Bearer test-user-jwt-token',
      'Idempotency-Key': uuidv4(), // Unique key per request
    },
  };

  const res = http.post(url, payload, params);

  check(res, {
    'status is 201 or 409': (r) => r.status === 201 || r.status === 409, // 201 = Reserved, 409 = Sold Out
    'response time < 80ms': (r) => r.timings.duration < 80,
  });

  sleep(0.1);
}
```